Instruções

Peso: 1,1 ponto | **Entrega:** digital, até 08/11 | **Foco:** Integrador final — arquivos, structs e ordenação

Contexto

Este é o checkpoint final e integrador do Quiz em C: ele consolida, em um único projeto, os blocos do segundo semestre — persistência em arquivos texto, structs e ordenação. O quiz passa a guardar os dados entre uma execução e outra e a modelar perguntas e jogadores como tipos próprios.

Descrição

O programa deve conter `struct Pergunta` e `struct Jogador`, organizados em vetores de structs. As perguntas são salvas em `perguntas.txt` e o histórico de partidas em `historico.txt`; ao iniciar, o programa recarrega os dados dos arquivos. O ranking de jogadores é ordenado do maior para o menor percentual de acertos com bubble sort, e o menu inclui opção para exibi-lo.

Requisitos obrigatórios

- `struct Pergunta` com enunciado, 4 alternativas e resposta correta.
- `struct Jogador` com nome, acertos, total de perguntas e percentual.

- Vetores de structs para perguntas e jogadores.
- Funções que recebem vetores como parâmetro.
- Salvar as perguntas em `perguntas.txt` e o histórico em `historico.txt` .
- Recarregar as perguntas e o histórico ao iniciar o programa.
- Verificação de abertura de arquivo contra NULL.
- Fechamento correto dos arquivos com `fclose` .
- Uso de `fscanf` com `%[^;]` para leitura de strings com espaços.
- Ranking ordenado por percentual de acertos (bubble sort).
- Menu com opção para exibir o ranking ordenado.
- Vídeo de demonstração de 2 minutos (tolerância: 1 min 30 s a 2 min 30 s) mostrando o programa rodando e explicando os conceitos usados.

Regra do vídeo

O vídeo é obrigatório: sem vídeo enviado, todo o trabalho individual é zerado (2,0 pontos da A2) — não só o CP3. Ele é a evidência de autoria e domínio do código — como não é possível auditar 100% do código entregue, a avaliação é feita sobre a sua explicação do próprio programa.

- Mostre o programa rodando: menu, cadastro de pergunta, uma rodada de jogo e o ranking ordenado.
- Demonstre a persistência: feche e reabra o programa (ou mostre os arquivos `perguntas.txt` e `historico.txt` gerados).
- Explique no código onde estão as `struct` s, a leitura/gravação de arquivo e o bubble sort — o que cada um faz.
- Vídeo que não mostra o programa em execução (só código no editor) não conta como vídeo enviado.
- Publique como vídeo não-listado (YouTube ou link de Drive) e registre o link no `README.md` — o arquivo de vídeo não entra no .zip. Teste o link em janela anônima antes de enviar.

Roteiro obrigatório do vídeo

Siga esta sequência — cada trecho é verificado na correção:

- **0–15 s — Compilar e abrir:** execute o comando de compilação na tela mostrando que compila sem erros; rode o programa e exiba o menu.
- **15–50 s — Fluxo principal:** cadastre 1 pergunta, jogue uma partida curta (2–3 perguntas) e mostre o placar com o percentual no fim.
- **50–70 s — Persistência:** abra `data/perguntas.txt` e `data/historico.txt` no editor mostrando o que foi gravado; feche e reabra o programa mostrando os dados carregados.
- **70–85 s — Ranking:** chame a opção de ranking mostrando a lista ordenada do maior para o menor percentual, com pelo menos 2 jogadores registrados.
- **85–120 s — Explicação no código:** no editor, aponte e explique com suas palavras as `struct` s (`Pergunta` e `Jogador`), uma função que recebe vetor, a leitura/gravação de arquivo e o bubble sort — diga o que cada um faz.

Preparação: terminal e editor abertos lado a lado (ou alterne entre eles); projeto compilável na hora; `perguntas.txt` e `historico.txt` já com alguns dados. Ensaie uma vez antes de gravar a versão final — 2 minutos passa rápido.

Dicas: fale o que está acontecendo enquanto executa ("aqui ele leu o arquivo e carregou as perguntas") — a explicação é o que comprova autoria; não precisa decorar nem ler código palavra por palavra — aponte o trecho e explique a ideia; grave em uma tomada só e não acelere a execução; sua voz narrando é obrigatória, mas você não precisa aparecer no vídeo.

Exemplo de comportamento esperado

Exemplo de arquivo `perguntas.txt` :

```
Qual a capital do Brasil?;Sao Paulo;Rio de Janeiro;Brasilia;Salvador;
Quanto e 5 + 3?;5;8;9;7;B
```

Exemplo de arquivo `historico.txt` :

```
Ana;3;5;60.00
Bruno;5;5;100.00
```

Dica: use `fprintf` para gravar e `fscanf` para ler. A função que carrega as perguntas pode receber o vetor e o tamanho como parâmetros para preencher os dados.

Estrutura de pastas esperada

A partir deste CP, o projeto deve seguir a seguinte estrutura de pastas:

```
meu-projeto-c/
├── src/
│   ├── main.c
│   ├── funcoes.c
│   └── modulo.c
├── include/
│   ├── funcoes.h
│   └── modulo.h
├── data/
│   ├── perguntas.txt
│   └── historico.txt
└── README.md
```

- `src/` : arquivos `.c` com o código-fonte.
- `include/` : arquivos `.h` com protótipos e constantes.
- `data/` : arquivos de dados utilizados em tempo de execução (quando houver).
- `README.md` : instruções de compilação, execução e descrição do projeto.

Compilação: `gcc -Wall -Wextra -Iinclude src/main.c src/funcoes.c src/modulo.c -o programa`

Restrições

- Não usar arrays paralelos — dados de uma mesma entidade vivem na mesma `struct` .
- Ordenação implementada manualmente (bubble sort), sem `qsort` da biblioteca

biblioteca.

Critérios de avaliação

- Código compila sem erros.
- Structs modelam perguntas e jogadores corretamente.
- Arquivos são abertos, lidos, gravados e fechados corretamente.
- Vetores passados como parâmetro às funções.
- Dados persistem entre execuções.
- Ranking ordenado corretamente em ordem decrescente.
- Tratamento de erro ao abrir arquivo.

Entrega

Compacte a pasta do projeto em .zip, incluindo o código-fonte, os arquivos de exemplo (`perguntas.txt` e `historico.txt`) e o `README.md` , e envie pelo Blackboard. O link do vídeo (não-listado) deve estar registrado no `README.md` — não inclua o arquivo de vídeo no .zip.